Parte A. Análisis y diseño

Necesidad	Estructura elegida	Operación principal	Complejidad esperada	Alternativa descartada	Justificación
pendientes en orden de registro	ArrayList<Paquete>	Agregar al final y recorren en orden	O(1)	LinkedList	Agregar al final no cambia la posición de los demás y linkedList gasta mas memoria
Consultar un paquete por código	HashMap<String, Paquete>	Containskey	O(1)	ArrayList	La consulta es la operación más repetida y recorrer todo O(n).
Municipios destino sin duplicados	HashSet<String>	add/ contains	O(1)	ArrayList	El set garantiza unicidad por construcción
Municipios ordenados y sin repetidos	TreeSet<String>	add y recorrido ordenado	O(logm)	HashSet	Mantiene siempre el orden sin repetidos, con hashset habría que ordenas cada vez
Atender por mayor prioridad	PriorityQueue<Paquete>	offer / poll	O(logn)	ArrayList	Siempre entrega el siguiente en despachar
Historial en orden de entrega	LinkedList<paquete> en la entrega sera lista simplemente enlazada propia	Agregar al final y recorrer	O(1)	ArrayDeque	Solo se agrega al final y se recorre en orden, el arraydeque no permite acceder por posición

Asociar código con su	Map<String, Paquete>	put/get	O(1)	TreeMap	No se necesita orden de claves y TreeMap costaría más por operación

Criterio	ArrayList<Paquete>	HashMap<String, Paquete>
Tiempo de búsqueda	O(n): En promedio realizará 250.000 comparaciones y 500.00 en el peor caso	O(1) = 1 o 2 dos comparaciones aproximadamente
Memoria adicional	Casi ninguna: un arreglo de referencias (~2 a 4 MB)	Tabla de ~1 millón de buckets (~4 MB) más un nodo por entrada (~32 B × 500.000 ≈ 16 MB): unos 20 MB extra, aproximado y según la JVM
Ventaja	Compacto, conserva el orden de registro y recorre rápido	Búsqueda casi constante, sin importar el tamaño
Desventaja	Cada consulta recorre todo, y el costo crece con n	Más memoria, sin orden, y el rehash al crecer cuesta O(n) de vez en cuando (se evita con capacidad inicial)

La estructura elegida es Hashmap <String, Paquete> ya que, como se especifica en el enunciado se deben realizar muchas búsqueda durante la jornada y se requiere que no se recorra todos los paquetes cada vez que se consulte por uno, con el arraylist para 500 mil paquetes tantas comparaciones por minuto satura la CPU, el hashmap hace un cálculo y una comparación siendo más rápido y eficientes. La memoria aproximada para este sería de 16 y 25 MB donde estas cantidades son despreciables y asumibles.

Parte C. Análisis algorítmico y pruebas

Análisis de algoritmos.

1. Divide y vencerás par obtener paquetes de mayor peso, donde con los 32 paquetes se ejecutan 63 llamadas, el resultado será PQ584 si llega a ocurrir un empate ganará el de la izquierda por el operador >=. La complejidad de la pila es de logn por esto puede procesar 1 millón de paquetes sin llegar a desbordarse. Una alternativa seria un ciclo simple donde la complejidad tambien es O(n) asi que no hay peor ni mejor caso
2. Búsqueda por código binario, este algoritmo tiene un costo de O(log n) en tiempo y O(1) en espacio, pero tiene la condición de que solo funciona sobre una lista previamente ordenada por código. Cada iteración compara un elemento del medio y descarta una mitad mediante el cálculo bajo + (alto - bajo) / 2. Su alternativa es la búsqueda con comparadores

3. Método recursivo, con cada llamada se avanza un índice, donde el caso base es cuando el índice es mayor o igual al tamaño; índice >= size. Su principal problema es que siempre recorre toda la lista por ende con listas muy grandes ocurre un desbordamiento

Analisis de estructuras de datos

1. Lista enlazada propia

Operación	Tiempo	Justificación
agregar	O(1)	Enlaza después de cola. Sin ella sería O(n).
eliminar	O(n)	La búsqueda es O(n). El reenlace es O(1).
obtener(i)	O(i), O(n) en el peor caso	No hay acceso directo.
tamaño y estaVacia	O(1)	Se lleva un contador.
Recorrido con iterador	O(n) en total	O(1) por paso.

2. Estructuras usadas en Centro de distribución

Estructura	Uso	Costo
HashMap	Consultar paquete por código	O(1) promedio
ArrayList	Registro y pendientes	add O(1) amortizado, get O(1), remove y contains O(n)
HashSet y TreeSet	Municipios sin repetir y ordenados	O(1) promedio y O(log m) por inserción
LinkedHashMap	Repartidores en orden de registro	O(1) por consulta

ArrayDeque	Cola de llegada	offer y poll O(1), remove(Object) O(n)
PriorityQueue	Despacho prioritario	offer y poll O(log n), peek O(1), remove(Object) O(n)

Tabla comparativa de costo en tiempo y en el espacio de las estructuras y algoritmos

Algoritmo	Tiempo	Espacio extra	Mejor caso	Peor caso
Divide y vencerás (mayor peso)	Θ(n)	O(log n)	Θ(n)	Θ(n)
Búsqueda binaria	O(log n)	O(1)	O(1)	O(log n)
Peso total (recursivo)	Θ(n)	O(n)	Θ(n)	Θ(n)
Conteo por prioridad (recursivo)	Θ(n)	O(n)	Θ(n)	Θ(n)
Ordenamiento (TimSort)	O(n log n)	O(n)	O(n)	O(n log n)
ListaEnlazada.agregar	O(1)	O(1)	O(1)	O(1)
ListaEnlazada.eliminar y obtener	O(n)	O(1)	O(1)	O(n)

Parte D. Uso justificado de inteligencia artificial

Parte	Herramienta	Qué preguntaron	Por qué la consultaron	Cómo verificaron la respuesta	Qué modificaron
A	Gemini	¿Por qué la clase Paquete implementa Comparable y no Comparator?	Confusión conceptual entre el orden natural y los criterios del comparator	se revisaron los requisitos del enunciado, verificando que el codigo define la identidad y orden por defecto del paquete	Se implementó el comparable<paquete> en la clase paquete
	Claude	Generación de las clases de prueba unitaria (JUnit 5) para el código existente, tomando como referencia el enunciado	El código ya estaba implementado y faltaba definir qué casos probar y cómo estructurar las pruebas	Ejecución de la suite en IntelliJ y contraste de los resultados con los ejemplos del enunciado (despacho PQ105, PQ731, PQ942, PQ420; peso total de Armenia 14.0 kg; mayor peso 18.7 kg)	Corrección de un identificador de método y de un nombre de clase que impedían compilar
	Claude	Verificación de que las pruebas no usaran operadores ternarios	Cumplir una convención de estilo definida por el equipo	Búsqueda del operador ?: en src/test y src/main	Ninguna, no se encontraron ocurrencias en las pruebas
	Claude	Análisis de tres métodos de CentroDistribucion sin pruebas por comportamiento incorrecto	Un análisis externo reportaba cumplimiento total, mientras que la lectura del código evidenciaba defectos	Revisión directa del código fuente de cada método	despacharPrioritario: condición corregida a != null. verSiguientePrioritario: uso de peek() en lugar de poll(). asignarPaquete: búsqueda en repartidores, registro de la asignación y

					cambio de disponibilidad
	Claude	Diagnóstico de errores de resolución de JUnit en el IDE	El pom.xml no declaraba la dependencia de pruebas	Recarga del proyecto Maven y comprobación de que los símbolos se resolvían	Se añadieron junit-jupiter 5.11.4 (alcance test) y maven-surefire-plugin 3.5.2
	Claude	Revisión del estado del repositorio frente a los requisitos del enunciado	Identificar incompatibilidades y funcionalidad pendiente antes de la entrega	Compilación del proyecto y ejecución completa de la suite	Alineación del nombre de método entre pruebas y código
	Claude	Pruebas para los métodos agregados posteriormente (entrega, historial, ordenamientos, búsqueda binaria, recursividad, divide y vencerás)	Cubrir la funcionalidad nueva sin modificar el código ya integrado	Ejecución de la suite completa (154 pruebas)	Se incorporó CentroDistribucionOperacionesTest
	Claude	Programa principal con menú de JOptionPane y datos iniciales mínimos	Demostrar cada operación con 30 paquetes, 5 destinos y 6 repartidores como mínimo	Ejecución del menú y de la demostración completa	(completar con los ajustes realizados)

D.1. Bitácora

D.2. Una respuesta cuestionable de IA

Respuesta original	Problema detectado	Análisis	Solución final
Un asistente de IA (Gemini) informó que todos los requisitos se cumplían y citó clases y métodos (ordenNatural(), PruebasListaEnlazada, uso de Collator, README)	Los elementos citados no existían en el repositorio en ese momento. Además, despacharProritario() tenía la condición invertida y verSiguientePrioritario() extraía el elemento en lugar de consultarlo	Comportamiento: la revisión evaluaba la descripción de las clases y no su ejecución. Restricción incumplida: el código no satisfacía el enunciado a pesar del informe de cumplimiento	Se corrigieron ambos métodos, se implementaron pruebas que fallan ante esos defectos y se contrastó cada afirmación del informe con el código real
Claude propuso un método buscarBinaria(codigo) que clonaba y ordenaba toda la lista con Collections.sort antes de aplicar la búsqueda binaria.	Aunque encuentra el elemento, ordenar la lista en cada búsqueda destruye la eficiencia del algoritmo y viola la precondición de trabajar sobre datos ya ordenados	Reordena innecesariamente la en cada consulta gastando así más memoria	Modificar el método para que reciba el reporte que ya cumple la precondición
Claude propuso guardar los paquetes pendientes en un ArrayList<Paquete>, validando si estaban pendientes con pendientes.contains(paquete) y eliminándolos tras la entrega con pendientes.remove(paquete).	ArrayList es ineficiente para buscar y eliminar por valor. Obliga a recorrer la lista elemento por elemento, colapsando el rendimiento si el volumen de paquetes crece	Estructura: ArrayList (inadecuada para búsquedas y eliminaciones por valor) Uso estándar de arreglo, pero con alto costo de CPU en iteraciones.	Validar la existencia y estado del paquete mediante una estructura con acceso directo como HashSet<String>